Laboratorio 2 - Arreglos y excepciones

Análisis

Requisitos funcionales

1. Solicitar al iniciar nombre del caso, código de identificación y detective responsable.
2. Permitir crear un nuevo caso que reemplace al actual y comience sin ubicaciones ni pistas.
3. Guardar hasta cinco ubicaciones en un arreglo básico de objetos Ubicacion.
4. Guardar en cada ubicación código, nombre, dirección o descripción, nivel de riesgo y estado.
5. Validar riesgo de 1 a 10 al crear o modificar una ubicación; lanzar IllegalArgumentException si no cumple.
6. Pedir al usuario la posición al registrar una ubicación y comprobar que esté dentro del arreglo.
7. Rechazar registros en posiciones ocupadas y validar la información antes de almacenarla.
8. Mantener null en los espacios libres y comprobarlo antes de utilizar un objeto del arreglo.
9. Consultar todas las ubicaciones existentes, mostrando su posición y omitiendo los null.
10. Consultar una ubicación por posición; informar si el índice es inválido o el espacio está vacío.
11. Modificar riesgo y estado únicamente de una ubicación existente.
12. Descartar una ubicación y devolver su posición al valor null.
13. Guardar una cantidad dinámica de pistas directamente en ArrayList de Pista, sin usar List.
14. Guardar en cada pista código, descripción, tipo, importancia y confiabilidad.
15. Validar importancia de 1 a 10 y confiabilidad de 0 a 100; lanzar IllegalArgumentException si no cumplen.
16. Registrar pistas sin permitir dos códigos iguales dentro del caso.
17. Consultar todas las pistas y mostrar un aviso cuando no existan.
18. **B**uscar una pista por código recorriendo el ArrayList.
19. Modificar la información de una pista existente y aplicar nuevamente las validaciones.
20. Localizar y eliminar del ArrayList la pista descartada o falsa.
21. Calcular cantidad de ubicaciones, espacios disponibles y ubicación de mayor riesgo.
22. Calcular cantidad de pistas, pista de mayor importancia, pista de mayor confiabilidad y promedio de importancia.
23. Usar solo los objetos registrados para los cálculos; evitar máximos o promedios que requieran elementos cuando no los hay.
24. Leer números con Scanner, capturar InputMismatchException y retirar la entrada incorrecta.
25. Manejar errores con try-catch y realizar al menos una acción final pertinente en finally.
26. Continuar funcionando después de los errores controlados.
27. Presentar el menú con sus trece opciones y terminar mediante la opción Salir.

1. ¿Qué propiedades y métodos tendrá cada clase?

Clase Ubicacion

Representa un lugar de investigación y protege la validez de su riesgo y estado.

Propiedad, tipo y modificadores	Propósito
codigo String private final	Identifica el lugar.
nombre String private final	Nombre reconocible del lugar.
direccion String private final	Dirección o descripción para localizarlo.
nivelRiesgo int private	Riesgo entero entre 1 y 10.
estado String private	Situación del lugar; texto no vacío, por ejemplo pendiente o investigada.

public Ubicacion(codigo: String, nombre: String, direccion: String, nivelRiesgo: int, estado: String)

Crea un lugar con código, nombre, dirección, riesgo y estado válidos.

private validarTexto(texto: String): String

Rechaza null o texto vacío; devuelve el texto sin espacios exteriores.

public actualizar(nivelRiesgo: int, estado: String): void

Valida riesgo y estado antes de asignarlos juntos. Lanza IllegalArgumentException si alguno es inválido.

public getNivelRiesgo(): int

Devuelve el riesgo para comparar ubicaciones en el reporte.

public toString(): String

Devuelve una descripción con los cinco datos del lugar.

Clase Pista

Representa una evidencia válida. Sus atributos son inmutables; al editarla, Caso reemplaza el objeto en la misma posición del ArrayList.

Propiedad, tipo y modificadores	Propósito
codigo String private final	Identifica la pista; único en el caso sin distinguir mayúsculas.
descripcion String private final	Explica la evidencia encontrada.
tipoEvidencia String private final	Categoría de evidencia, como física, digital o testimonial.
nivelImportancia int private final	Relevancia entera entre 1 y 10.
nivelConfiabilidad int private final	Confiabilidad entera entre 0 y 100.

public Pista(codigo: String, descripcion: String, tipoEvidencia: String, nivelImportancia: int, nivelConfiabilidad: int)

Crea una pista. Valida textos, importancia de 1 a 10 y confiabilidad de 0 a 100.

private validarTexto(texto: String): String

Rechaza null o texto vacío y devuelve el texto recortado.

public getCodigo(): String

Devuelve el código para buscar y evitar duplicados.

public getNivelImportancia(): int

Devuelve la importancia para máximos y promedio.

public getNivelConfiabilidad(): int

Devuelve la confiabilidad para calcular su máximo.

public toString(): String

Devuelve una descripción con los cinco datos de la evidencia.

Clase Caso

Administra una investigación: conserva sus datos, el arreglo de ubicaciones y el ArrayList de pistas. Ejecuta las operaciones y los cálculos sin leer la consola.

Propiedad, tipo y modificadores	Propósito
nombre String private final	Título de la investigación.
codigo String private final	Identificación del caso.
detective String private final	Nombre del responsable.
ubicaciones arreglo de Ubicacion private final	Arreglo fijo de cinco Ubicacion; cada espacio disponible contiene null.
pistas ArrayList de Pista private final	Colección dinámica que almacena únicamente objetos Pista válidos.

public Caso(nombre: String, codigo: String, detective: String)

Valida nombre, código y detective; crea un arreglo de cinco null y un ArrayList vacío.

private validarTexto(texto: String): String

Valida textos del caso y códigos de búsqueda; devuelve el texto recortado.

private validarPosicion(posicion: int): void

Comprueba el rango 0 a 4 antes de acceder al arreglo; lanza IndexOutOf**B**oundsException fuera del rango.

public registrarUbicacion(posicion: int, ubicacion: Ubicacion): void

Recibe índice y objeto válido; rechaza un índice inválido, una posición ocupada o un objeto null. Guarda el objeto.

public consultarUbicacion(posicion: int): Ubicacion

Recibe un índice y devuelve el objeto existente. Rechaza índices inválidos y posiciones vacías.

public listarUbicaciones(): String

Recorre el arreglo y devuelve solo ubicaciones no null con sus índices, o un aviso si no hay ninguna.

public modificarUbicacion(posicion: int, riesgo: int, estado: String): void

Recibe posición, riesgo y estado; obtiene la ubicación existente y actualiza sus valores validados.

public descartarUbicacion(posicion: int): void

Comprueba que exista la ubicación y asigna null a esa posición.

private buscarIndicePista(codigo: String): int

Recorre el ArrayList comparando códigos sin distinguir mayúsculas; devuelve el índice o -1.

public registrarPista(pista: Pista): void

Recibe una pista válida; rechaza null y códigos repetidos, luego ejecuta add.

public buscarPista(codigo: String): Pista

Recibe un código y devuelve la pista encontrada; lanza IllegalArgumentException si no existe.

public listarPistas(): String

Devuelve el texto de todas las pistas o un aviso si la colección está vacía.

public modificarPista(codigoActual: String, nuevaPista: Pista): void

Recibe el código actual y un nuevo objeto validado; comprueba existencia y unicidad del código nuevo, luego usa set en el índice original.

public eliminarPista(codigo: String): void

Localiza el código y usa remove(indice); rechaza códigos inexistentes.

public contarUbicaciones(): int

Recorre el arreglo y cuenta los elementos distintos de null.

public contarEspaciosDisponibles(): int

Devuelve cinco menos la cantidad de ubicaciones registradas.

public ubicacionMayorRiesgo(): Ubicacion

Devuelve la ubicación con máximo riesgo o null si no hay ninguna. Omite los espacios vacíos.

public contarPistas(): int

Devuelve el tamaño actual del ArrayList.

public pistaMayorImportancia(): Pista

Recorre las pistas y devuelve la de mayor importancia o null si no existen.

public pistaMayorConfiabilidad(): Pista

Recorre las pistas y devuelve la de mayor confiabilidad o null si no existen.

public promedioImportancia(): double

Suma las importancias en double y divide por size(). Rechaza una colección vacía con IllegalStateException.

public generarReporte(): String

Devuelve identificación del caso y los siete indicadores. Con pistas vacías no calcula promedio y muestra avisos.

Clase Main

Driver del programa. Contiene main(), solicita los datos iniciales, presenta el menú y coordina las operaciones del caso activo.

Propiedad, tipo y modificadores	Propósito
entrada Scanner private static final	Scanner compartido por los métodos estáticos para leer System.in.

public static main(args: arreglo de String): void

Punto de entrada. args contiene los argumentos de ejecución (no se utilizan). Mantiene el ciclo y captura errores de las operaciones; cierra Scanner en finally.

private static leerTexto(mensaje: String): String

Recibe el mensaje de solicitud; repite la lectura hasta obtener un texto no vacío.

private static leerEntero(mensaje: String): int

Recibe el mensaje; usa nextInt(), captura InputMismatchException, limpia la línea en finally y reintenta.

private static leerCaso(): Caso

Solicita nombre, código y detective; devuelve una nueva instancia de Caso.

private static leerUbicacion(): Ubicacion

Solicita los cinco datos de un lugar y devuelve una Ubicacion validada.

private static leerPista(): Pista

Solicita los cinco datos de evidencia y devuelve una Pista validada.

private static mostrarMenu(): void

Imprime las trece opciones del sistema.

2. ¿Qué tipos tendrán las propiedades y los métodos?

Los textos usan String; los niveles, índices y cantidades usan int; el promedio usa double. Las operaciones de modificación retornan void. Las consultas retornan String, Ubicacion o Pista según su resultado. Main usa Scanner y main recibe arreglo de String. Las firmas completas se indican en la respuesta 1 y en el UML.

3. ¿Qué propiedad usa un arreglo básico?

Caso.ubicaciones tiene tipo arreglo de Ubicacion y se inicializa con un arreglo de cinco ubicaciones. Los índices válidos son 0, 1, 2, 3 y 4. Los espacios no registrados contienen null.

4. ¿Qué propiedad usa ArrayList?

Caso.pistas tiene tipo ArrayList de Pista y se inicializa vacía. Se usa ArrayList directamente; no se declara ni importa List.

5. ¿Qué modificadores de visibilidad se emplean?

Todos los atributos son private. Los constructores de las clases del modelo y las operaciones necesarias desde otras clases son public. Las validaciones, la búsqueda por índice y los lectores auxiliares son private. Los métodos de Main y su Scanner son static. Los atributos final no pueden reasignarse; el contenido del arreglo y del ArrayList sí puede cambiar.

6. ¿Qué parámetros requiere cada método?

Los constructores reciben los datos del objeto. Las operaciones sobre ubicaciones reciben una posición int; registrar recibe además Ubicacion y modificar recibe riesgo int y estado String. Las búsquedas y eliminaciones de pistas reciben código String; modificar recibe código actual y Pista nueva. Los lectores reciben el mensaje String. Los métodos de listado y cálculo no necesitan parámetros. Las firmas de la respuesta 1 especifican los parámetros de cada método.

7. ¿Cómo se inicializan y validan los objetos?

Se usan constructores. Se rechazan textos null o vacíos, riesgo y importancia fuera de 1 a 10 y confiabilidad fuera de 0 a 100. Caso valida índice, disponibilidad y unicidad. Ubicacion.actualizar valida ambos datos antes de asignar. Para editar una pista se construye un objeto válido y solo después se reemplaza el anterior mediante set; un error deja intactos los datos previos.

8. ¿Cómo se identifica null?

Se comprueba si el elemento del arreglo es distinto de null antes de usar sus métodos. Para una consulta individual se valida primero el índice y después se comprueba si la posición está vacía. Si no hay ubicación, se informa mediante una IllegalStateException controlada en main.

9. ¿Cómo se busca, modifica y elimina en ArrayList?

buscarIndicePista recorre con for y compara los códigos mediante equalsIgnoreCase, retirando espacios exteriores. Retorna un índice o -1. La consulta usa get; la modificación, set; la eliminación, remove(indice). Antes de modificar se comprueba que el nuevo código no pertenezca a otra pista.

10. ¿Qué excepciones se manejan y dónde?

leerEntero captura InputMismatchException y limpia la línea en finally. El ciclo de main captura IllegalArgumentException (datos y códigos), IndexOutOf**B**oundsException (índices) e IllegalStateException (posiciones vacías u ocupadas). Un try exterior captura NoSuchElementException si se cierra la entrada y su finally cierra Scanner. Se previenen accesos null y división por cero en lugar de depender de esos errores.

Diseño - Diagrama de clases

- privado; + público; subrayado: static; {readOnly}: final.

crea

Main

- entrada: Scanner {readOnly}

+ main(args: String[]): void
- leerTexto(mensaje: String): String
- leerEntero(mensaje: String): int
- leerCaso(): Caso
- leerUbicacion(): Ubicacion
- leerPista(): Pista
- mostrarMenu(): void

usa

Caso

- nombre: String {readOnly}
- codigo: String {readOnly}
- detective: String {readOnly}
- ubicaciones: Ubicacion[] {readOnly}
- pistas: ArrayList<Pista> {readOnly}

+ Caso(nombre: String, codigo: String, detective: String)
- validarTexto(texto: String): String
- validarPosicion(posicion: int): void
+ registrarUbicacion(posicion: int, ubicacion: Ubicacion): void
+ consultarUbicacion(posicion: int): Ubicacion
+ listarUbicaciones(): String
+ modificarUbicacion(posicion: int, riesgo: int, estado: String): void
+ descartarUbicacion(posicion: int): void
- buscarIndicePista(codigo: String): int
+ registrarPista(pista: Pista): void
+ buscarPista(codigo: String): Pista
+ listarPistas(): String
+ modificarPista(codigoActual: String, nuevaPista: Pista): void
+ eliminarPista(codigo: String): void
+ contarUbicaciones(): int
+ contarEspaciosDisponibles(): int
+ ubicacionMayorRiesgo(): Ubicacion
+ contarPistas(): int
+ pistaMayorImportancia(): Pista
+ pistaMayorConfiabilidad(): Pista
+ promedioImportancia(): double
+ generarReporte(): String

0..5

ubicaciones

Ubicacion

- codigo: String {readOnly}
- nombre: String {readOnly}
- direccion: String {readOnly}
- nivelRiesgo: int
- estado: String

+ Ubicacion(codigo: String, nombre: String, direccion: String, nivelRiesgo: int, estado: String)
- validarTexto(texto: String): String
+ actualizar(nivelRiesgo: int, estado: String): void
+ getNivelRiesgo(): int
+ toString(): String

0..*

pistas

Pista

- codigo: String {readOnly}
- descripcion: String {readOnly}
- tipoEvidencia: String {readOnly}
- nivelImportancia: int {readOnly}
- nivelConfiabilidad: int {readOnly}

+ Pista(codigo: String, descripcion: String, tipoEvidencia: String, nivelImportancia: int, nivelConfiabilidad: int)
- validarTexto(texto: String): String
+ getCodigo(): String
+ getNivelImportancia(): int
+ getNivelConfiabilidad(): int
+ toString(): String

ubicaciones: arreglo fijo de 5 objetos.
Posiciones válidas: 0 a 4.
Las posiciones vacías contienen null.
pistas: ArrayList de objetos Pista.
Riesgo e importancia: 1 a 10.
Confiabilidad: 0 a 100.
Los códigos de pista no se repiten.
Main contiene el método main().
Las líneas discontinuas son dependencias.
Las líneas continuas son asociaciones.

crea